juliacon

MLThermoProperties.jl: Hybrid Models for Thermodynamic Property Prediction in Julia

Sebastian Schmitt[1], Hans Hasse[1], and Fabian Jirasek[1]

[1]Laboratory of Engineering Thermodynamics (LTD), RPTU Kaiserslautern

ABSTRACT

We present MLThermoProperties.jl, a Julia package that provides a variety of state-of-the-art thermodynamic models that combine modern machine learning methods with physical knowledge. These hybrid models obey hard physical constraints while being more accurate and applicable to a wider scope of substances than established models. MLThermoProperties.jl is built upon the Clapeyron.jl package, leveraging its rich thermodynamic solver ecosystem. The MLThermoProperties.jl models significantly improve molecular property prediction in various applications in science and engineering, e.g., chemical process engineering, and can be coupled with Julia's rich ecosystem for scientific modeling and simulation.

Keywords

Julia, Thermodynamics, Machine Learning, Chemical Engineering

1. Introduction

Knowledge of thermodynamic properties of fluids is crucial in many fields of engineering and science, e.g., for developing new chemical and biotechnological processes, optimizing heat pumps, or designing carbon capture and storage technologies. However, experimental data on thermodynamic properties are notoriously scarce due to the high cost and effort of measurements, making reliable prediction models indispensable. In recent years, machine learning (ML) has emerged as a particularly promising approach to thermodynamic modeling [2].

These models, despite being developed using methods from machine learning, are thermodynamically consistent, i.e., they obey hard physical constraints. This consistency is either achieved by an appropriate architecture [13, 6] or by exploiting existing thermodynamic models to form new hybrid models [15, 4, 5]. Based only on the SMILES code of a substance [17] – a textual representation of the molecular structure – the MLPROP models predict multiple thermodynamic properties, outperforming established models in both accuracy and scope [6]. Among the covered properties are phase equilibria of pure substances and mixtures [14], which are central to chemical process design, as well as transport properties such as diffusion coefficients [15] that govern molecular mass transfer. The models utilize different ML architectures, including the chemical language model ChemBERTa [1] and molecular graph neural networks.

In Julia, a rich ecosystem for thermodynamics and chemical engineering already exists. 'Clapeyron.jl' [16], a mature and comprehensive thermodynamic package, is a central part of this ecosystem. It combines a large number of thermodynamic models (including equations of state (EoS) and Gibbs excess energy (g^E) models) with advanced and efficient solvers. Leveraging Julia's excellent extensibility, several packages extend 'Clapeyron.jl', e.g., 'EntropyScaling.jl' [12] for modeling transport properties, or 'Langmuir.jl' [11] for modeling adsorption.

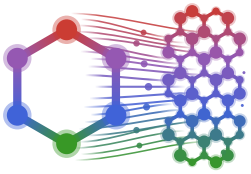


Fig. 1: Logo of 'MLThermoProperties.jl'.

This paper introduces 'MLThermoProperties.jl', which provides Julia implementations of hybrid thermodynamic models, based on 'Clapeyron.jl' – enabling the prediction of thermodynamic properties for any substance in Julia. 'MLThermoProperties.jl' not only provides implementations of existing models, but also serves as a central anchor point for the development of new models. Due to the excellent extensibility of Julia packages, 'MLThermoProperties.jl' substantially broadens the capabilities for modeling chemical processes, based on state-of-the-art molecular property prediction. The individual models are presented and their integration into the existing thermodynamic ecosystem is explained. Additionally, illustrative applications of these models in chemical process simulations are showcased using packages from the wider scientific-modeling ecosystem in Julia, particularly from the SciML organization, e.g., 'ModelingToolkit.jl' and 'DifferentialEquations.jl'.

2. Package overview

MLThermoProperties.jl implements several hybrid models and makes them accessible to the thermodynamic solvers from Clapeyron.jl. This integration simplifies the implementation of new models and enables making use of the existing thermodynamic solvers already implemented in Clapeyron.jl.

2.1 Models

All currently implemented models were developed under the umbrella of MLPROP [8]. The following models are currently implemented and validated against their original publication:

HANNA Hard-constraint neural network for the excess Gibbs energy g^E of liquid mixtures, from which activity coefficients are obtained in a thermodynamically consistent manner [13]. Only the SMILES of the components and the temperature are required as input. Two versions are available: 'ogHANNA', the original model

for binary mixtures [13], and ‘HANNA‘ (alias ‘multHANNA‘), which is trained on VLE and LLE data and applicable to multi-component mixtures [7].

UNIFAC 2.0 Machine-learning-enhanced versions of the group-contribution methods UNIFAC and mod. UNIFAC (Dortmund) for predicting activity coefficients [4, 3]. Missing group-interaction parameters are completed by matrix completion, which extends the applicability and accuracy of the methods. Both are implemented directly in ‘Clapeyron.jl‘.

GRAPPA Graph neural network for the vapor pressure of pure components, predicting the parameters of the Antoine equation and thereby also boiling points [5]. The SMILES of the component is the only input.

ESE Hybrid model for binary diffusion coefficients at infinite dilution that incorporates the Stokes-Einstein equation, ensuring a physically consistent temperature dependence [15]. It requires the SMILES of solute and solvent as well as a viscosity model for the solvent.

ChemBERTa Chemical language model that encodes a SMILES into a molecular embedding [1], which serves as molecular input representation in HANNA. It is provided by the registered package ‘ChemBERTa.jl‘ and can also be used independently.

2.2 Integration in Julia’s Thermodynamic Ecosystem

All models are implemented by extending a property function from Clapeyron.jl or EntropyScaling.jl and making use of existing thermodynamic solvers. This is exemplarily explained for the `HANNA` model and the `ESE` model in the following. For the Gibbs excess energy model `HANNA`, a custom function `Clapeyron.excess_gibbs_free_energy(model::HANNA,p,T,z)` is defined which calculates the Gibbs excess energy G^E through a neural network (see Ref. [6]). This is the only thermodynamic property that needs to be defined. Derived quantities like the activity coefficients γ_i and the excess enthalpy H^E can be calculated based on the HANNA model by Clapeyron’s property functions, i.e. `Clapeyron.activity_coefficient(model::HANNA,p,T,z)` and `Clapeyron.excess(model::HANNA,p,T,z,enthalpy)`, without further ado. This exploits the relation between those properties which can be calculated as derivations of G^E, i.e.

$$\ln \gamma_i = \frac{1}{RT} \left(\frac{\partial G^E}{\partial n_i} \right)_{T,p,n_{j\neq i}} \tag{1}$$

$$H^E = -RT^2 \left(\frac{\partial \left(G^E/RT \right)}{\partial T} \right)_{p,\boldsymbol{n}} . \tag{2}$$

The derivatives are evaluated by automatic differentiation, so no analytical expressions have to be supplied. Note that the excess enthalpy is calculated as a mixing property, i.e., the pure-component model passed via the `puremodel` keyword must provide residual properties; a cubic EoS such as `PR` is sufficient.
The `ESE` model follows the same principle, but extends `EntropyScaling.jl` instead. The only method implemented evaluates the Stokes-Einstein equation corrected by a factor b_{ij}, which a neural network ensemble predicts from molecular descriptors of solute i and solvent j [15], i.e.

$$D_{ij}^{\infty} = b_{ij} \, \frac{k_\mathrm{B} T}{6\pi \, \eta_j(p,T) \, r_i} \tag{3}$$

with the Boltzmann constant k_B and the hydrodynamic radius r_i of the solute. Since b_{ij} depends on the components only, it is evaluated once when the model is constructed and the temperature dependence is dictated entirely by the Stokes-Einstein equation, which enforces physical consistency. The solvent viscosity η_j is obtained by calling `EntropyScaling.viscosity(model,p,T)` on a viscosity model supplied by the user, so that any model of `EntropyScaling.jl` can be combined with `ESE`, e.g., a substance-specific reference correlation (`RefpropRES`), a group-contribution entropy scaling model (`GCES`), or a constant experimental viscosity (`ConstantModel`). Based on this single method, `EntropyScaling.inf_diffusion_coefficient` is readily available for `ESE`, returning the diffusion coefficients of all component pairs or of an individual solute-solvent pair.

2.3 Extending the Package

The same mechanism is the intended way to add new models. Listing 1 shows a complete minimal g^E model: a struct holding the components, the model parameters, and the pure-component models, together with a single method for the excess Gibbs energy. Nothing else is required – activity coefficients, excess properties, and phase equilibria are immediately available through the property functions and solvers of Clapeyron.jl. In the models of MLThermoProperties.jl, the parameters are the weights of a neural network and the returned expression is its forward pass, but the interface is the same.

Code 1: A minimal g^E model extending Clapeyron.jl.

```julia
using Clapeyron
using Clapeyron: ActivityModel, EoSVectorParam,
                 init_puremodel, Rgas

struct MyGE{P} <: ActivityModel
    components::Vector{String}
    params::Vector{Float64}
    puremodel::EoSVectorParam{P}
    references::Vector{String}
end

function MyGE(comps; params, puremodel=PR)
    pure = init_puremodel(puremodel, comps,
                          String[], false)
    return MyGE(comps, params, pure, String[])
end

function Clapeyron.excess_gibbs_free_energy(
        model::MyGE, p, T, z)
    x = z ./ sum(z)
    A, B = model.params
    return sum(z)*Rgas(model)*T*x[1]*x[2]*(A+B/T)
end
```

2.4 Validation and Performance

All models are validated against the reference implementations of the original publications in the test suite of the package, which calls the original Python code through PythonCall.jl. For HANNA, the activity coefficients of the Julia and the Python implementation agree within a maximum relative deviation of $1.4 \cdot 10^{-6}$ over the tested systems; the Antoine parameters predicted by GRAPPA agree with the published values within a relative tolerance of 10^{-4}. The residual deviations are of the order of the single-precision arithmetic used in the neural networks.

Table 1. : Execution times of typical calls for the system tetrachloromethane + *tert*-butanol at 298.15 K with HANNA and GRAPPA, measured on an Intel Core Ultra 5 235U with Julia 1.13.

Call	Time
Excess Gibbs energy G^E	245 μs
Activity coefficients γ_i	577 μs
Excess enthalpy H^E	467 μs
Vapor pressure p^s (GRAPPA)	0.1 μs
Bubble pressure	1.2 ms
Model construction (once)	53 ms

Table 1 lists execution times of typical calls for the binary system tetrachloromethane + *tert*-butanol at 298.15 K. The cost of the graph neural network of GRAPPA is paid once when the model is constructed, since it predicts the Antoine parameters of the pure components; every subsequent vapor-pressure evaluation is then a simple correlation. The evaluation of HANNA, in contrast, enters every solver iteration and accounts for roughly half of the time of a bubble-point calculation. With about one millisecond per bubble point, the models are fast enough to be called inside iterative flowsheet solvers or the right-hand side of a differential equation.

3. Example Application

3.1 Calculating Mixture Properties

The following example computes the isothermal p-xy diagram of the system tetrachloromethane (1) + *tert*-butanol (2) at $T = 298.15$ K, using `HANNA` for the activity coefficients and `GRAPPA` for the pure-component vapor pressures (Listing 2). Both models are constructed from the component names only, which are resolved to SMILES through the Clapeyron.jl database. The vapor-liquid equilibrium is calculated with the γ-φ approach; assuming an ideal gas phase and neglecting the Poynting correction, it reduces to the extended Raoult's law

$$p\, y_i = x_i\, \gamma_i(\boldsymbol{x}, T)\, p_i^{\mathrm{s}}(T), \tag{4}$$

where x_i and y_i are the mole fractions of component i in the liquid and the vapor phase, respectively. The activity coefficients γ_i are supplied by HANNA and the vapor pressures p_i^{s} by GRAPPA, while the phase-equilibrium solver is taken from Clapeyron.jl. Since the pure-component model is a free choice, GRAPPA can be replaced by a cubic or multiparameter EoS without changing the rest of the code. The resulting phase diagram is shown in Figure 2.

Code 2: Calculation of the p-xy diagram of tetrachloromethane (1) + *tert*-butanol (2) at 298.15 K with HANNA and GRAPPA.

```
using MLThermoProperties, Clapeyron, CairoMakie

comps = ["tetrachloromethane", "tert-butanol"]
model = multHANNA(comps; puremodel=GRAPPA)

T = 298.15
x₁ = range(0.0, 1.0, 51)
p, y₁ = zeros(51), zeros(51)
for (i, x) in enumerate(x₁)
    z = [x, 1-x]
    pᵢ, _, _, yᵢ = bubble_pressure(model, T, z)
    p[i], y₁[i] = pᵢ/1e5, yᵢ[1]
end

fig = Figure(size=(400, 280), fontsize=14)
xlab = L"x_1,\,y_1 \;/\; \mathrm{mol\,mol^{-1}}"
ax = Axis(fig[1,1]; limits=((0,1), nothing),
    xlabel=xlab, ylabel=L"p \;/\; \mathrm{bar}")
lines!(ax, x₁, p; color=:blue, label="bubble")
lines!(ax, y₁, p; color=:red, label="dew")
axislegend(ax; position=:lt, framevisible=false)
save(joinpath(@__DIR__, "pxy_diagram.pdf"), fig)
```

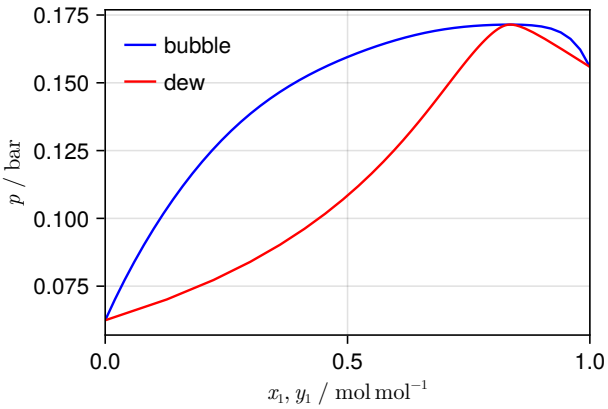


Fig. 2: Predicted p-xy diagram of tetrachloromethane (1) + *tert*-butanol (2) at $T = 298.15$ K, obtained with HANNA (activity coefficients) and GRAPPA (pure-component vapor pressures).

3.2 Diffusion Coefficients

Transport properties are accessed in the same way, but through EntropyScaling.jl. Listing 3 computes the diffusion coefficients at infinite dilution of ethanol in *n*-decane and of *n*-decane in ethanol between 280 K and 340 K. The `ESE` model is again built from the component names alone; the solvent viscosity required by the Stokes-Einstein equation is taken from the default `RefpropRES` correlations, which can be exchanged for a group-contribution model or a constant experimental value through the `vismodel` keyword. The two curves in Figure 3 differ by about one order of magnitude, which reflects the different viscosities of the two solvents.

Code 3: Diffusion coefficients at infinite dilution in the system ethanol + *n*-decane with ESE (plotting omitted).

```
using MLThermoProperties, EntropyScaling
using CoolProp, CairoMakie

model = ESE(["ethanol", "n-decane"])

p = 1e5
T = range(280.0, 340.0, 40)
D₁₂ = inf_diffusion_coefficient.(model, p, T;
    solute="ethanol", solvent="n-decane")
D₂₁ = inf_diffusion_coefficient.(model, p, T;
    solute="n-decane", solvent="ethanol")
```

3.3 Fitting Classical Models

Established process simulators expect the parameters of classical g^E models rather than neural networks. The predictions of the ML models can therefore be condensed into such a model by a parameter fit. Listing 4 fits the three parameters τ_{12}, τ_{21}, and α of the NRTL model [10] to the activity coefficients predicted by HANNA for tetrachloromethane (1) + *tert*-butanol (2) at $T = 298.15$ K, minimizing

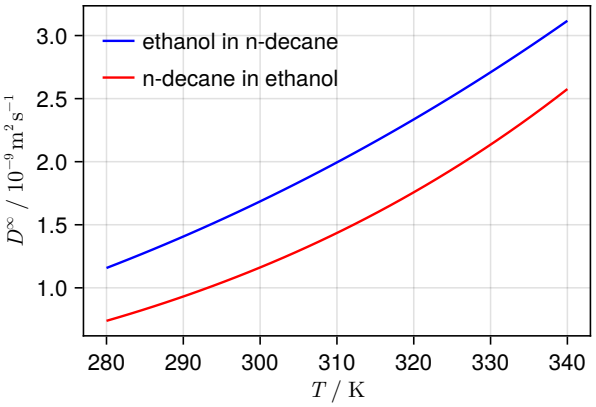


Fig. 3: Predicted diffusion coefficients at infinite dilution of ethanol in *n*-decane and of *n*-decane in ethanol at $p = 1$ bar, obtained with ESE and the `RefpropRES` viscosity correlations.

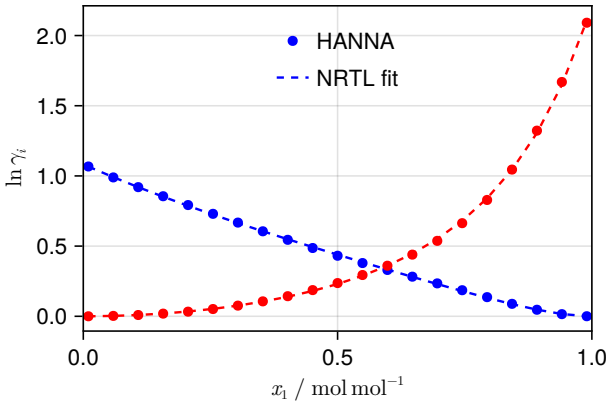


Fig. 4: Logarithmic activity coefficients of tetrachloromethane (1) + *tert*-butanol (2) at $T = 298.15$ K predicted by HANNA (symbols) and the fitted NRTL model (lines).

$$\mathcal{L} = \frac{1}{2K} \sum_{k=1}^{K} \sum_{i=1}^{2} \left(\ln \gamma_i^{\mathrm{NRTL}} \left(x_i^k, T \right) - \ln \gamma_i^{\mathrm{pred}} \left(x_i^k, T \right) \right)^2 \quad (5)$$

over $K = 21$ compositions. Both the ML model and NRTL are Clapeyron.jl models, so the same `activity_coefficient` call serves both the prediction and the fit, and the optimizer is taken from Optim.jl [9]. The fit yields $\tau_{12} = 1.873$, $\tau_{21} = 0.447$, and $\alpha = 0.578$ and reproduces the predictions of HANNA within 0.021 in $\ln \gamma_i$ (Figure 4).

Code 4: Fit of the NRTL model to activity coefficients predicted by HANNA (plotting omitted).

```julia
using MLThermoProperties, Clapeyron
using Optim, CairoMakie

comps = ["tetrachloromethane", "tert-butanol"]
T, p = 298.15, 1e5
x₁ = range(0.01, 0.99, 21)
γ(m, x) = activity_coefficient(m, p, T, [x, 1-x])

hanna = multHANNA(comps)
lnγ_pred = [log.(γ(hanna, x)) for x in x₁]

nrtl = NRTL(comps)
function loss(θ)
    nrtl.params.a.values .= [0.0 θ[1]; θ[2] 0.0]
    nrtl.params.c.values .= [0.0 θ[3]; θ[3] 0.0]
    err = sum(sum(abs2, lnγ .- log.(γ(nrtl, x)))
              for (x, lnγ) in zip(x₁, lnγ_pred))
    return err / (2*length(x₁))
end

res = optimize(loss, [1.0, 1.0, 0.3], NelderMead())
θ = Optim.minimizer(res)
loss(θ)
```

3.4 Chemical Process Simulation

4. Conclusions

MLThermoProperties.jl makes hybrid thermodynamic models that combine machine learning with physical knowledge available in Julia. By implementing each model as an extension of a single property function of Clapeyron.jl or EntropyScaling.jl, the models inherit the complete solver ecosystem of these packages: activity coefficients, excess properties, and phase equilibria follow from one machine-learned expression for the excess Gibbs energy, and derived properties are obtained by automatic differentiation without analytical derivatives. The same mechanism keeps the package open for new models, which require little more than a struct and one method.

The models implemented here are also accessible through the MLPROP web interface [8], which is aimed at quick interactive calculations of single systems. MLThermoProperties.jl complements this by making the same models available inside automated workflows: they can be called in loops, embedded in process models, fitted to, differentiated through, and combined with the wider Julia ecosystem for scientific modeling and simulation, as demonstrated above.

5. Acknowledgements

6. References

[1] W. Ahmad, E. Simon, S. Chithrananda, G. Grand, and B. Ramsundar. ChemBERTa-2: Towards Chemical Foundation Models, 2022. doi:10.48550/arXiv.2209.01712.

[2] H. Hasse, S. Schmitt, and F. Jirasek. Artificial intelligence in thermodynamics: hybrid modeling of thermophysical properties of fluids. *Current Opinion in Chemical Engineering*, 51:101236, 2026. doi:10.1016/j.coche.2026.101236.

[3] N. Hayer, H. Hasse, and F. Jirasek. Modified UNIFAC 2.0-A Group-Contribution Method Completed with Machine Learning. *Industrial & Engineering Chemistry Research*, 64(20):10304–10313, 2025. doi:10.1021/acs.iecr.5c00077.

[4] N. Hayer, T. Wendel, S. Mandt, H. Hasse, and F. Jirasek. Advancing thermodynamic group-contribution methods by machine learning: UNIFAC 2.0. *Chemical Engineering Journal*, 504:158667, 2025. doi:10.1016/j.cej.2024.158667.

[5] M. Hoffmann, H. Hasse, and F. Jirasek. GRAPPA—A hybrid graph neural network for predicting pure component vapor pressures. *Chemical Engineering Journal Advances*, 22:100750, 2025. doi:10.1016/j.ceja.2025.100750.

[6] M. Hoffmann, T. Specht, Q. Göttl, J. Burger, S. Mandt, H. Hasse, and F. Jirasek. A machine-learned expression for the excess Gibbs energy, 2025. doi:10.48550/arXiv.2509.06484.

[7] M. Hoffmann, T. Specht, Q. Göttl, J. Burger, S. Mandt, H. Hasse, and F. Jirasek. Thermodynamically consistent machine learning model for excess Gibbs energy. *Nature Communications*, 17(1):3485, 2026. doi:10.1038/s41467-026-71430-y.

[8] M. Hoffmann, T. Specht, N. Hayer, H. Hasse, and F. Jirasek. MLPROP - An Interactive Web Interface for Thermophysical Property Prediction with Machine Learning. *Chemie Ingenieur Technik*, 97(11-12):1052–1056, 2025. doi:10.1002/cite.70004.

[9] Patrick K. Mogensen and Asbjørn N. Riseth. Optim: A mathematical optimization package for Julia. *Journal of Open Source Software*, 3(24):615, 2018. doi:10.21105/joss.00615.

[10] H. Renon and J. M. Prausnitz. Local compositions in thermodynamic excess functions for liquid mixtures. *AIChE Journal*, 14(1):135–144, 1968. doi:10.1002/aic.690140124.

[11] V. V. Santana, A. Riedemann, T. Christiansson, and P. J. Walker. Langmuir.jl.

[12] S. Schmitt, A. Riedemann, E. B. D. M. Alves, and LéoSchmider. se-schmitt/EntropyScaling.jl: v0.3.5, 2025. doi:10.5281/zenodo.17097480.

[13] T. Specht, M. Nagda, S. Fellenz, S. Mandt, H. Hasse, and F. Jirasek. HANNA: hard-constraint neural network for consistent activity coefficient prediction. *Chemical Science*, 15:19777–19786, 2024. doi:10.1039/D4SC05115G.

[14] T. Specht, M. Nagda, S. Fellenz, S. Mandt, H. Hasse, and F. Jirasek. HANNA: Hard-constraint Neural Network for Consistent Activity Coefficient Prediction, 2024. doi:10.48550/arXiv.2407.18011.

[15] J. Wagner, Z. Romero, K. Münnemann, S. Schmitt, T. Specht, H. Hasse, and F. Jirasek. Hybrid Machine Learning for Enhanced Prediction of Diffusion Coefficients in Liquids, 2026. doi:10.48550/arXiv.2603.02761.

[16] P. J. Walker, H.-W. Yew, and A. Riedemann. Clapeyron.jl: An Extensible, Open-Source Fluid Thermodynamics Toolkit. *Industrial & Engineering Chemistry Research*, 61(20):7130–7153, 2022. doi:10.1021/acs.iecr.2c00326.

[17] D. Weininger. SMILES, a chemical language and information system. 1. Introduction to methodology and encoding rules. *Journal of Chemical Information and Computer Sciences*, 28(1):31–36, 1988. doi:10.1021/ci00057a005.